



SINGLETON PATTERN

1. Mục tiêu bài học

Sau bài này, sinh viên có thể:

- Hiểu bản chất Singleton
- Áp dụng Singleton cho:
 - Configuration, Logger
 - In-memory Cache, Event Bus
- Nhận diện rủi ro Singleton với Multithreading
- Hiểu mối quan hệ Singleton ↔ DI ↔ Clean Architecture

2. Đặt vấn đề – Khi nào cần?

Trong hệ thống Order Management, có những đối tượng:

- Chỉ nên tồn tại một instance duy nhất.
- Được dùng xuyên suốt hệ thống.
- Không gắn với nghiệp vụ cụ thể của 1 order.

Ví dụ: Logger, AppConfig, Cache.

❓ Nếu tạo mới mỗi lần dùng thì sao?

- Lãng phí tài nguyên (Memory leak).
- Trạng thái không nhất quán (Inconsistent state).

3. Ví dụ sai – Không dùng Singleton

✗ Vấn đề: Không chia sẻ state, không kiểm soát vòng đời.

```
public class OrderService
{
    public void PlaceOrder(Order order)
    {
        // Mỗi lần gọi là một lần tạo mới
        var logger = new Logger();
        logger.Log("Order placed");
    }
}
```

4. Singleton Pattern – Định nghĩa

Định nghĩa (GoF):

Singleton đảm bảo một class chỉ có duy nhất một instance, và cung cấp điểm truy cập toàn cục tới instance đó.

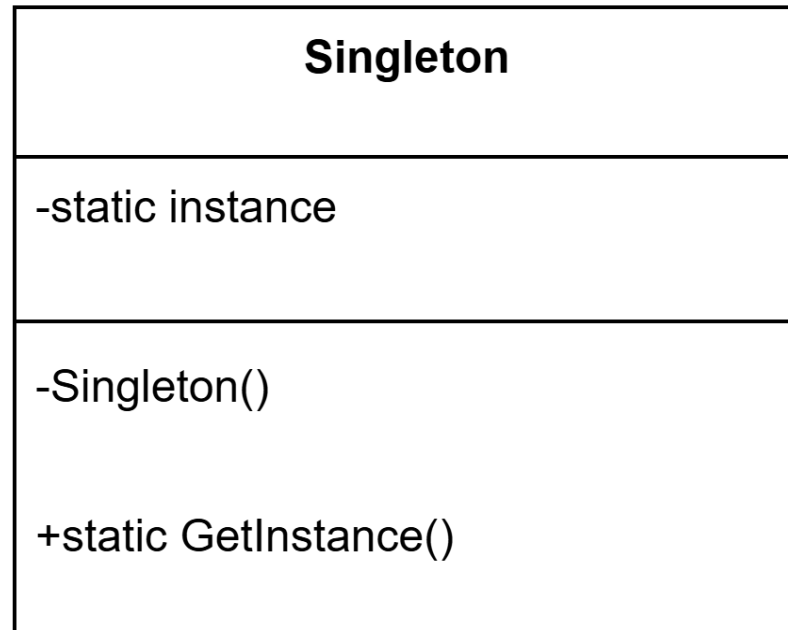
2 ý quan trọng:

1. One instance (Duy nhất)
2. Global access (Truy cập toàn cục)

5. UML – Singleton Pattern

✚ Constructor private: Ngăn chặn **new** từ bên ngoài.

✚ Instance static: Được quản lý bên trong class.



6. C# – Singleton cơ bản (Lazy Initialization)

```
public class Logger
{
    private static Logger _instance;
    private Logger() { } // Private Constructor
    public static Logger Instance
    {
        get
        {
            if (_instance == null)
                _instance = new Logger();
            return _instance;
        }
    }
    public void Log(string msg) =>
        Console.WriteLine(msg);
}
```

7. Áp dụng trong Order Management

```
public class OrderService
{
    public void PlaceOrder(Order order)
    {
        // Gọi qua property static
        Logger.Instance.Log("Order placed");
    }
}
```

8. Rủi ro Thread Safety

- Trong môi trường đa luồng (Web API), đoạn code trên có thể tạo nhiều instance.

Vấn đề nghiêm trọng: Thread Safety

```
if (_instance == null)
{
    // Thread A dừng ở đây
    // Thread B chạy vào và tạo instance
    // Thread A chạy tiếp và tạo instance thứ 2
    _instance = new Logger();
}
```

9. Singleton Thread-safe (Double-Check Locking)

```
private static readonly object _lock = new object();
```

```
public static Logger Instance {  
    get {  
        if (_instance == null) {  
            lock (_lock) {  
                if (_instance == null)  
                    _instance = new Logger();  
            }  
        }  
        return _instance;  
    }  
}
```

10. Singleton chuẩn nhất C# – Lazy<T>

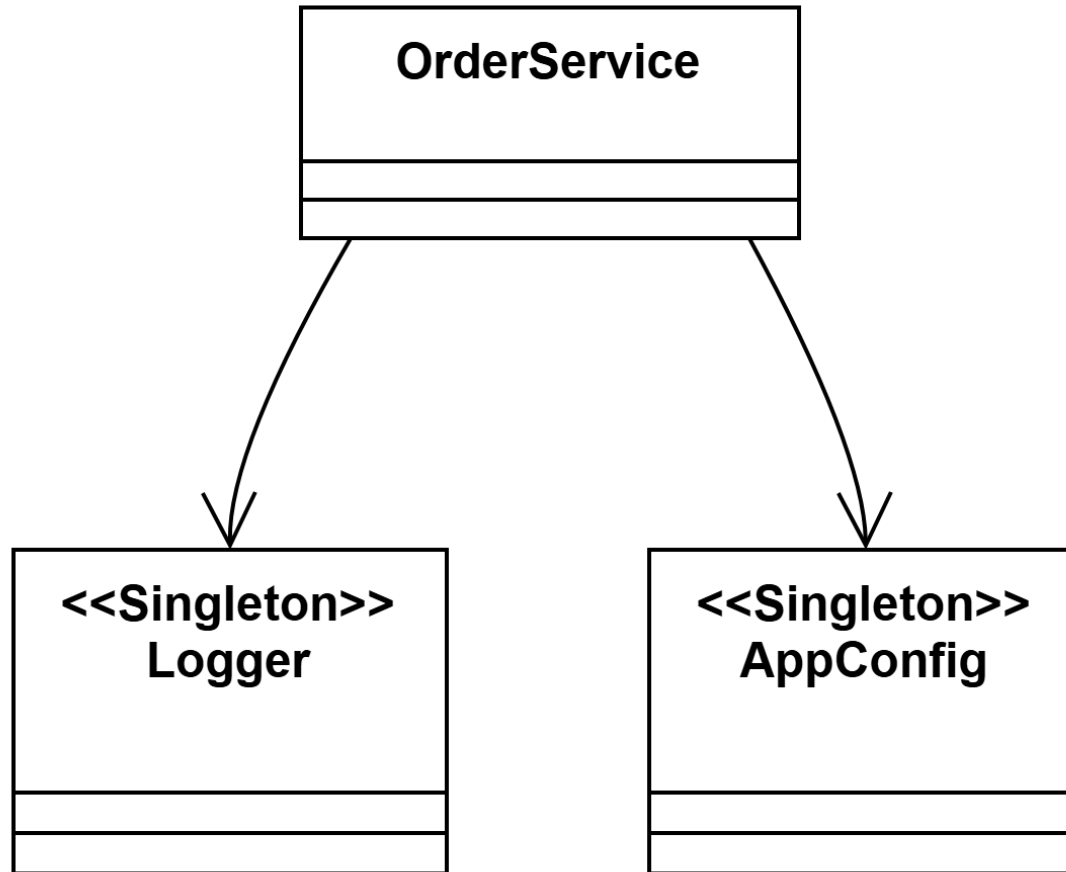
```
public class AppConfig
{
    // Thread-safe & Lazy by default

    private static readonly Lazy<AppConfig> _instance =
        new Lazy<AppConfig>(() => new AppConfig());

    private AppConfig() { Console.WriteLine("Load config"); }

    public static AppConfig Instance => _instance.Value;
}
```

11. UML – Singleton trong Order System



13. Khi nào NÊN dùng Singleton?

✓ NÊN dùng khi:

- Logger (Ghi log tập trung)
- Configuration (Load 1 lần dùng mãi)
- Cache (In-memory caching)
- Event Bus (Kênh giao tiếp)
- Connection Pool

📌 Đặc điểm chung:

- Stateless hoặc state dùng chung cho toàn app.
- Không chứa logic nghiệp vụ thay đổi liên tục.

14. Khi nào KHÔNG nên dùng?

✗ KHÔNG nên dùng khi:

- Order, User, Payment (Đây là Domain Entities)
- Business Service có state riêng theo request

📌 Tại sao?

- Singleton sống quá lâu (bằng tuổi đời ứng dụng).
- Khó giải phóng bộ nhớ.
- Gây khó khăn khi viết Unit Test (Global state).

15. Singleton vs Dependency Injection

Tiêu chí	Singleton Pattern (Cổ điển)	DI Singleton (Hiện đại)
Quản lý vòng đời	Class tự quản lý	DI Container (IoC)
Truy cập	Global (Static)	Explicit Dependency (Constructor)
Testability	Khó mock/fake	Dễ (Mock Interface)
Khuyến dùng	Hạn chế	Rất khuyến khích

16. Singleton trong ASP.NET Core

📌 DI Container là cách hiện đại thay thế Singleton thuần.

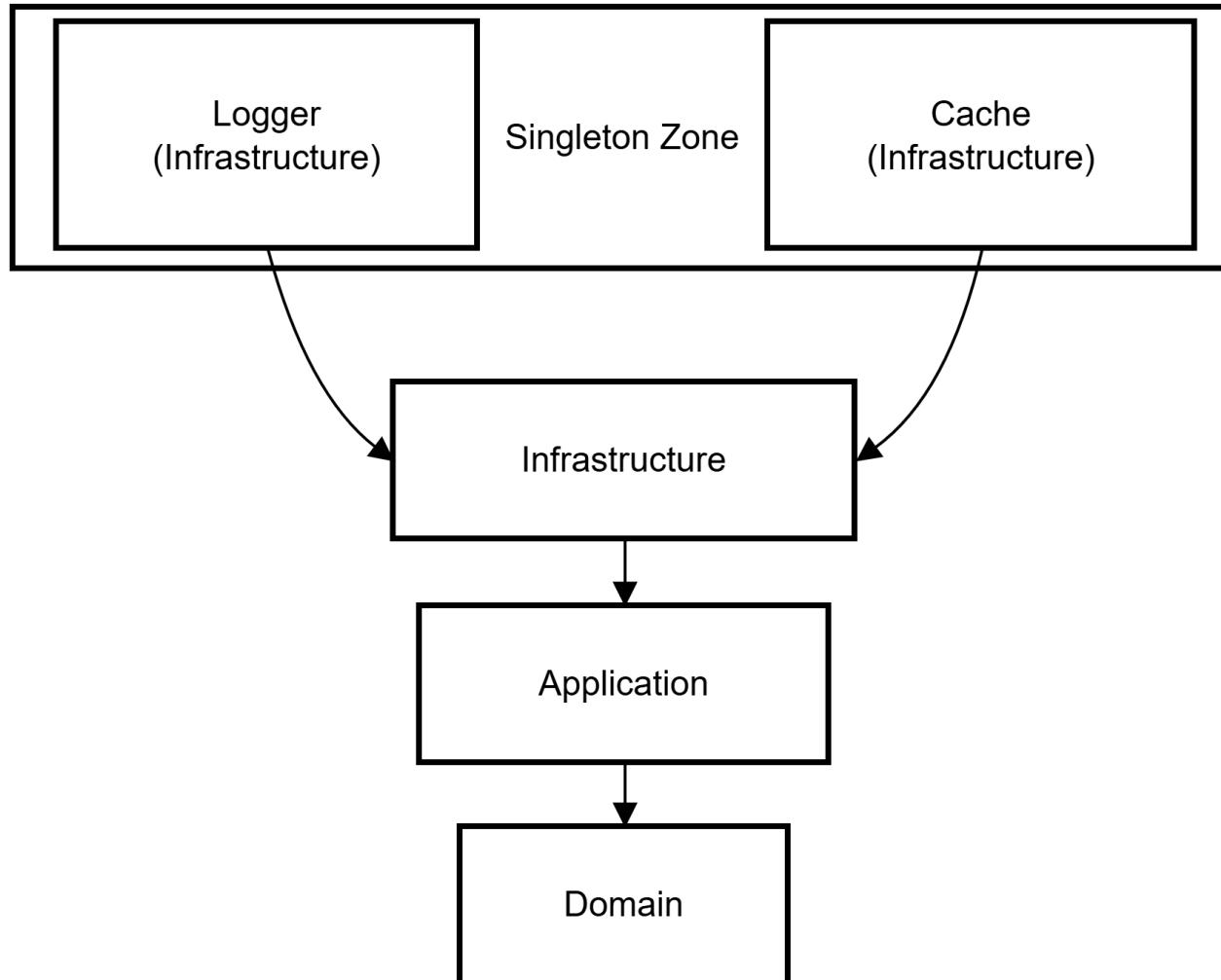
```
// Program.cs  
// Container sẽ tạo instance duy nhất và inject  
vào nơi cần
```

```
services.AddSingleton<ILogger, Logger>();  
services.AddSingleton<IAppConfig, AppConfig>();
```

```
// Usage  
public OrderService(ILogger logger) { ... }
```

17. Liên hệ Clean Architecture

📌 Singleton thường nằm ở Infrastructure. Không đặt Singleton trong Domain.



18. Bài tập trên lớp



Bài 1:

Viết Singleton OrderCache lưu tổng số đơn hàng trong ngày.
Đảm bảo thread-safe.



Bài 2:

So sánh Singleton thủ công vs DI Singleton (AddSingleton).



Bài 3:

Refactor đoạn code sử dụng `Logger.Instance` thành `ILogger` (DI).

19. Thông điệp

“ Singleton không xấu.

- Lạm dụng Singleton mới là vấn đề. ”

“ Dùng Singleton cho hạ tầng.

- Đừng dùng cho nghiệp vụ. ”